CAHIER DES CHARGES · ÉNONCÉ ÉTUDIANT

LE DUEL DES DEUX SORCIERS

Réaliser un jeu de combat en console avec la programmation orientée objet en Python.

GANDALF
100 PV · 3 attaques · XP

VS

ROI MORT D'ANGMAR
100 PV · 3 attaques · XP

Une arène. Deux personnages. Un seul vainqueur.

4 h 30
DURÉE TOTALE · PAUSES COMPRISES

Python 3
LANGAGE / ENVIRONNEMENT VS CODE

Console
INTERFACE TEXTE / TOUR PAR TOUR

Votre mission : concevoir les objets du jeu, implémenter les règles de combat et faire fonctionner une partie complète. Vous construirez ce programme progressivement, avec le formateur.

> **Important.** Ce document fixe ce que votre programme doit faire et la manière dont ses classes doivent collaborer. Il ne contient pas la solution Python. Les diagrammes UML servent de plan de réalisation.

01 / LE BESOIN

Un combat de console, piloté par des objets

Gandalf affronte le Roi Mort d'Angmar. Le joueur choisit à chaque tour l'une des trois attaques de Gandalf. Le Roi Mort, contrôlé par l'ordinateur, choisit automatiquement l'une des siennes. Une attaque peut réussir ou être esquivée. En cas de réussite, les points de vie de la cible diminuent et l'attaquant progresse en expérience.

La partie commence avec **100 points de vie (PV) par personnage et 0 point d'expérience (XP)**. Elle se termine dès qu'un des deux combattants atteint 0 PV. Le programme affiche alors le nom du vainqueur.

Exemple de partie (les dégâts réels dépendent des coups et de l'esquive)

```
=== GANDALF CONTRE LE ROI MORT D'ANGMAR ===
Gandalf lève son bâton. Vous jouez Gandalf !
Le Roi Mort d'Angmar avance dans l'ombre.

--- Tour 1 ---
1. Coup de bâton (12 dégâts, 10 % d'esquive)
2. Éclair magique (20 dégâts, 25 % d'esquive)
3. Flamme d'Anor (32 dégâts, 50 % d'esquive)
Votre choix (1, 2 ou 3) : 1
Gandalf utilise Coup de bâton : 12 dégâts !
Gandalf gagne 10 XP.
Le Roi Mort choisit Coup d'épée.
Le Roi Mort utilise Coup d'épée : Gandalf esquive !

Gandalf : 100/100 PV | 10 XP | x1.10
Roi Mort d'Angmar : 88/100 PV | 0 XP | x1.00
```

Cette transcription illustre un enchaînement possible : le hasard peut produire une autre issue, tout en respectant les mêmes règles.

Ce que fait le joueur

Il lance le programme, choisit un coup entre 1 et 3 et observe l'évolution du duel.

Ce que fait le programme

Il gère les tours, les probabilités d'esquive, les dégâts, les gains d'XP et la victoire.

02 / CE QUE LE JEU DOIT PERMETTRE

Fonctionnalités obligatoires

RF-01	**Démarrer une partie.** Le programme crée Gandalf et le Roi Mort d'Angmar, affiche leur présentation et initialise le numéro du tour à 1.
RF-02	**Proposer trois attaques.** Chaque personnage possède exactement trois coups, chacun défini par un nom, des dégâts de base et une probabilité d'esquive.
RF-03	**Permettre le choix du joueur.** Le joueur saisit 1, 2 ou 3. Une valeur invalide provoque un nouveau choix sans interrompre la partie ni consommer un tour.
RF-04	**Jouer automatiquement le Roi Mort.** L'ordinateur choisit aléatoirement une de ses trois attaques.
RF-05	**Résoudre l'esquive.** Chaque attaque réalise un tirage aléatoire ; si la cible esquive, elle ne perd aucun PV et l'attaquant ne gagne aucun XP.
RF-06	**Calculer les dégâts.** Quand le coup réussit, appliquer le coefficient de puissance du personnage attaquant, retirer les PV et attribuer 10 XP.
RF-07	**Afficher l'état du duel.** Faire apparaître pour chaque combattant les PV restants, les XP et le coefficient actuel.
RF-08	**Terminer au bon moment.** Le combat cesse immédiatement à 0 PV. Un personnage vaincu ne peut jamais riposter.
RF-09	**Protéger l'état.** Les PV ne peuvent pas être négatifs ; les dégâts négatifs et les coups impossibles doivent être rejetés.
RF-10	**Identifier un vainqueur unique.** En fin de partie, afficher le résultat, les états finaux et le personnage gagnant.

Périmètre du projet. Pas d'interface graphique, de sauvegarde, de multijoueur ni de système de déplacement. L'objectif est la qualité de la modélisation objet et la fiabilité du combat.

03 / LES RÈGLES QUI GOUVERNENT L'ARÈNE

Attaques, esquive et expérience

Les coups sont caractérisés par leurs **dégâts de base** et par la **chance qu'a la cible d'esquiver ce coup**. La probabilité dépend du coup choisi, et non d'une compétence permanente du défenseur. L'esquive est automatique : elle ne constitue pas une quatrième action.

Combattant	Attaque	Dégâts de base	Esquive de la cible
Gandalf	Coup de bâton	12	10 %
	Éclair magique	20	25 %
	Flamme d'Anor	32	50 %
Roi Mort	Coup d'épée	14	10 %
	Lame de Morgul	22	30 %
	Souffle d'effroi	30	45 %

Évolution de la puissance

coefficient = min(2, 1 + XP / 100)

dégâts infligés = int(dégâts de base × coefficient)

Un coup qui réussit utilise les XP **déjà acquis**. Les **10 XP gagnés après ce coup** ne renforcent donc que le coup suivant. Le coefficient est limité à ×2 ; le stock d'XP, lui, peut continuer à augmenter.

XP avant le coup	Coefficient	Coup de bâton : dégâts réussis
0	×1,00	12
10	×1,10	13
20	×1,20	14
100	×2,00	24
150	×2,00	24

> **À ne pas confondre :** une attaque esquivée rapporte 0 XP et 0 dégât. Les PV ne descendent jamais sous 0, même si les dégâts sont supérieurs à la vie restante.

04 / DIAGRAMME DE CAS D'UTILISATION

Qui fait quoi dans le système ?

Le seul acteur humain est le **joueur**, qui incarne Gandalf. Le Roi Mort n'est pas un second joueur : son comportement est géré par le programme.

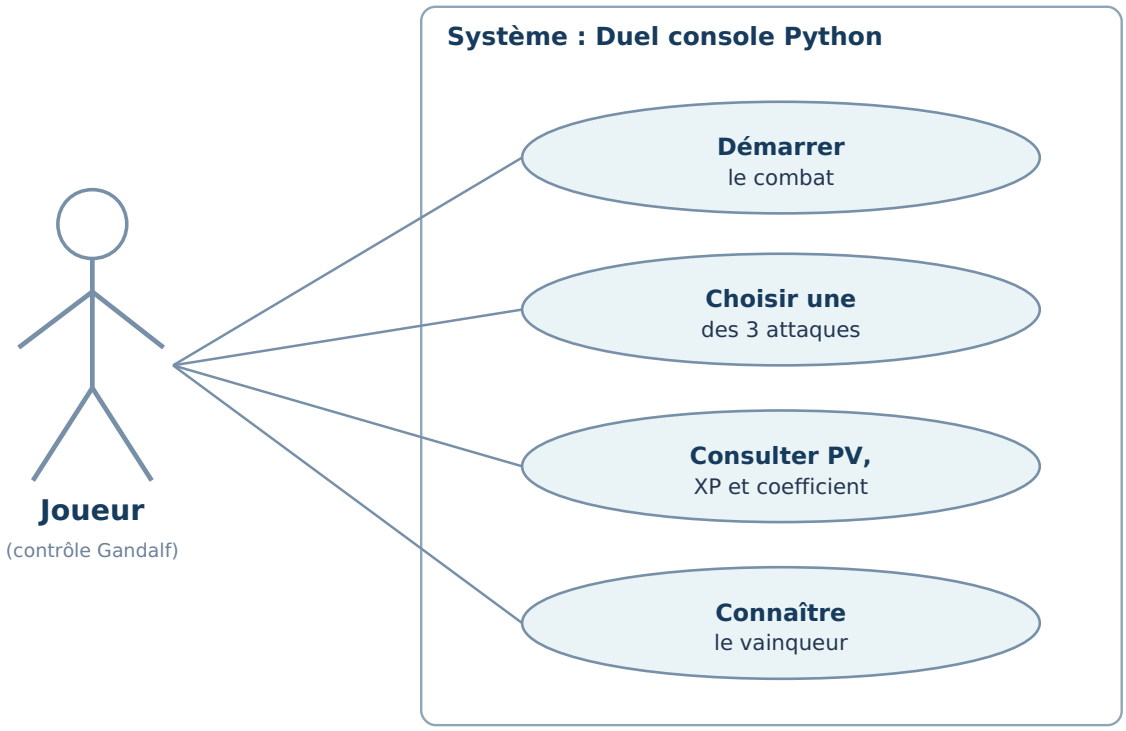


Diagramme UML de cas d'utilisation — interactions visibles depuis le point de vue du joueur.

Question de conception. Dans votre code, qui doit recevoir le choix de l'utilisateur ? Qui doit calculer les dégâts ? Ces deux responsabilités doivent-elles appartenir à la même classe ?

Frontière du système

Le système commence lorsque le joueur lance **main.py** et finit après l'annonce du vainqueur. Toutes les décisions automatiques (tirage d'esquive, attaque du Roi Mort, suivi des tours) sont internes au programme.

05 / DIAGRAMME DE CLASSES

La structure attendue du programme

Les deux combattants partagent les mêmes mécanismes : PV, XP, attaque et réception des dégâts. Ils possèdent toutefois des listes de coups et des présentations différentes. Une classe séparée organise la partie.

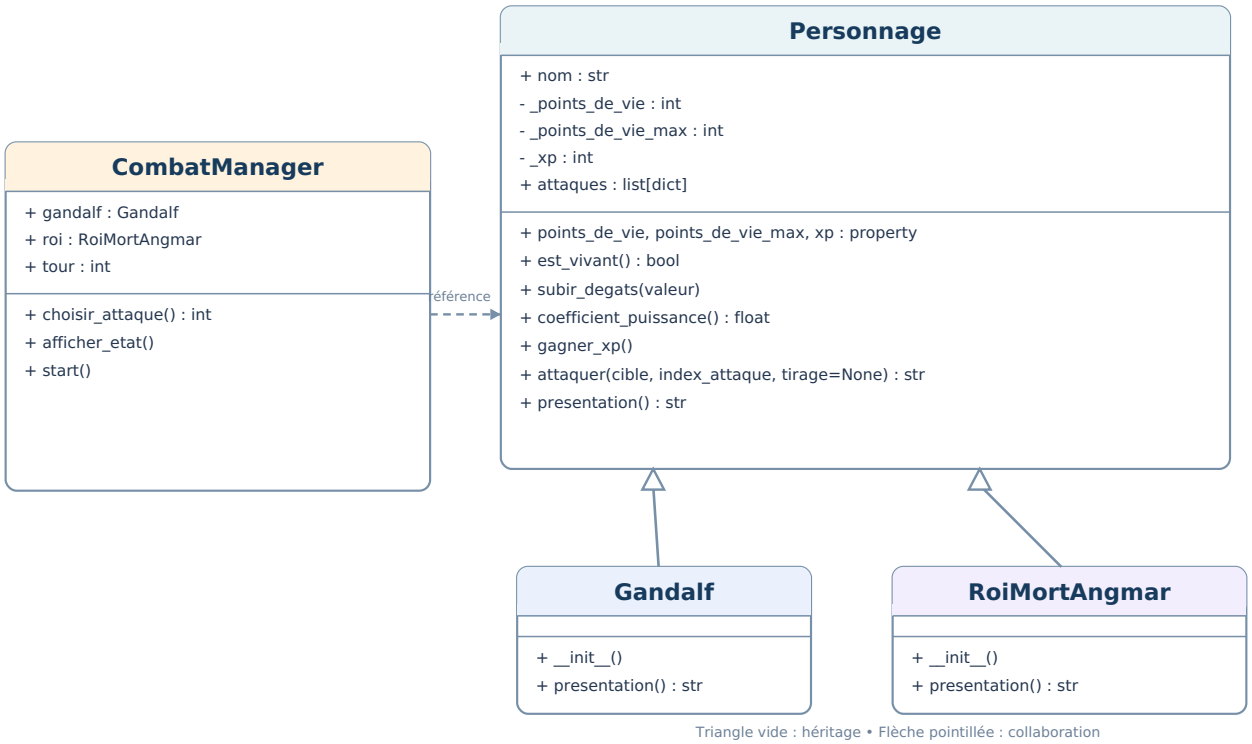


Diagramme UML de classes — membres principaux du projet final.

Comment lire ce schéma ? Le signe **+** identifie un membre public ; le signe **−** représente ici les attributs internes, matérialisés en Python par un nom commençant par `_`. Le triangle vide indique l'**héritage**. Une propriété `@property` rend une valeur consultable sans autoriser directement sa modification.

> **Point important.** Les attaques sont représentées par une **liste de dictionnaires** : il n'est pas demandé de créer une classe `Attaque`. Le diagramme correspond à l'architecture du cours.

06 / DIAGRAMME DE SÉQUENCE

Comment deux objets collaborent pendant une attaque

Ce diagramme représente un appel à `attaquer()`. La méthode appartient à l'attaquant, mais **les dégâts sont appliqués par l'objet cible**. C'est une règle importante de répartition des responsabilités en POO.

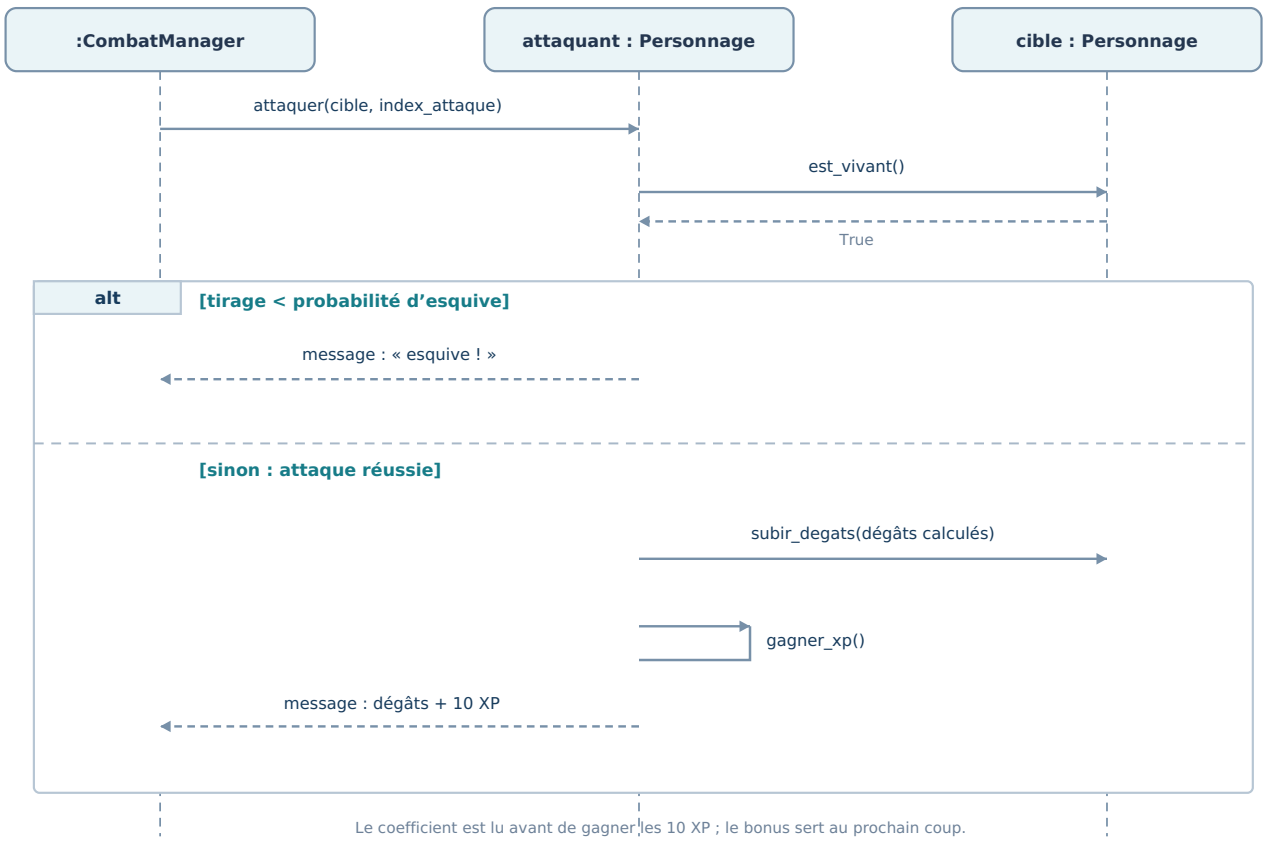


*Diagramme de séquence UML — branche esquivée et branche réussie (cadre **alt**).*

À retenir. En cas d'esquive, aucun appel à `subir_degats()` ou `gagner_xp()` ne doit être effectué. En cas de réussite, les dégâts sont calculés avant l'ajout des XP.

07 / DIAGRAMME D'ACTIVITÉ

Comment se déroule un combat complet

Le combat est une répétition de tours orchestrée par `CombatManager.start()`. Gandalf attaque en premier à chaque tour. Si son attaque achève le Roi Mort, celui-ci ne joue pas.

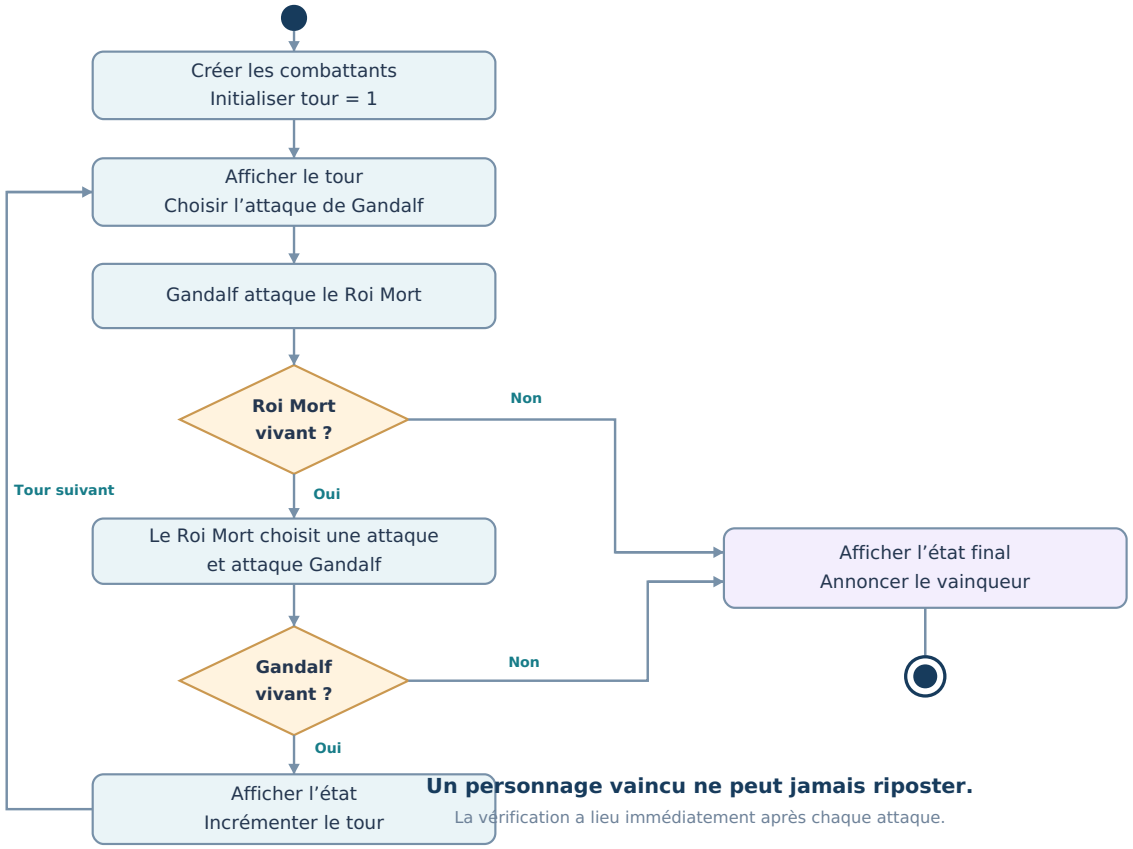


Diagramme d'activité UML — déroulement des tours, décisions et sorties de boucle.

> **Incontournable.** Contrôler la survie du Roi Mort *avant* d'organiser sa riposte. Contrôler ensuite la survie de Gandalf avant de commencer le tour suivant.

08 / CONTRAINTES D'ARCHITECTURE

Un fichier, une responsabilité identifiable

```
combat_python/
├── main.py
├── combat_manager.py
├── personnage.py
├── gandalf.py
├── roi_mort_angmar.py
└── test_combat.py
```

Fichier	Rôle attendu
`main.py`	Point d'entrée uniquement : instancier `CombatManager`, puis appeler sa méthode `start()`.
`combat_manager.py`	Créer les deux personnages, afficher le menu, valider la saisie, décider des tours, afficher l'état et la victoire.
`personnage.py`	Porter les règles communes : PV, XP, esquive, coefficient, dégâts, attaque, vérification de vie.
`gandalf.py`	Classe héritée de `Personnage`, configuration des trois attaques de Gandalf et présentation personnalisée.
`roi_mort_angmar.py`	Classe héritée de `Personnage`, configuration des trois attaques du Roi Mort et présentation personnalisée.
`test_combat.py`	Vérifications simples et reproductibles des règles principales.

Contraintes techniques

- **CT-01.** Utiliser Python 3 et le terminal intégré de VS Code ; aucune bibliothèque externe n'est nécessaire.
- **CT-02.** Le lancement doit fonctionner avec `python main.py` (ou `python3 main.py` selon la machine).
- **CT-03.** Le code doit illustrer `class`, `__init__`, `self`, `@property`, `super()` et la redéfinition de `presentation()`.
- **CT-04.** Les PV et l'XP ne doivent pas être modifiés arbitrairement par `CombatManager` : celui-ci utilise les méthodes des personnages.
- **CT-05.** La logique du duel n'est pas écrite dans `main.py` : la classe `CombatManager` est obligatoire.

> **Liberté limitée.** Vous pouvez choisir le style exact des messages de console. En revanche, les noms des classes, les responsabilités, les règles de combat et les valeurs des six attaques doivent rester conformes à cet énoncé.

09 / COMMENT VÉRIFIER LA CONFORMITÉ ?

Cas de test à réussir

Un programme qui s'exécute une fois sans erreur n'est pas forcément correct. Les scénarios suivants doivent être reproductibles. Pour tester l'esquive sans dépendre du hasard, la méthode `attaquer()` peut accepter un paramètre de tirage facultatif, comme indiqué dans le diagramme de classes.

ID	Situation donnée	Résultat attendu
T01	Début du jeu	100 PV, 0 XP, coefficient ×1 pour chaque personnage.
T02	Gandalf utilise « Coup de bâton » à 0 XP ; le coup réussit	Le Roi Mort passe à 88 PV, Gandalf à 10 XP.
T03	Gandalf réussit un deuxième « Coup de bâton »	Le Roi Mort passe de 88 à 75 PV (13 dégâts) ; Gandalf atteint 20 XP.
T04	Une attaque est esquivée	0 dégât et 0 XP gagné ; les PV ne changent pas.
T05	Le personnage possède 100 XP ou plus	Coefficient plafonné à ×2.
T06	Il ne reste que 5 PV à la cible ; elle subit 12 dégâts	Ses PV deviennent 0, jamais −7.
T07	Le joueur saisit « 0 », « abc » ou « 4 »	Message d'erreur et nouvelle demande ; aucune attaque déclenchée.
T08	Gandalf porte un coup mortel	Le Roi Mort ne riposte pas ; le jeu annonce Gandalf vainqueur.
T09	Un personnage vaincu tente d'attaquer	Action rejetée explicitement.
T10	On essaie de retirer des dégâts négatifs	Action rejetée explicitement.

> **Votre méthode de vérification :** pour chaque test, noter les valeurs initiales, l'action réalisée et les valeurs finales observées. Une erreur corrigée doit être retestée.

10 / ORGANISATION ET LIVRABLES

Déroulement de la réalisation — 4 h 30

La construction sera guidée par le formateur. À chaque étape, le programme devra rester exécutable : n'ajoutez pas toutes les règles avant d'avoir vérifié les précédentes.

09:00–09:15	**Comprendre et modéliser.** Identifier les objets, leurs états et leurs responsabilités ; lire les diagrammes UML.
09:15–09:50	**Créer les premières classes.** Déclarer `Personnage` et `CombatManager`, initialiser les deux personnages et appeler `start()`.
09:50–10:25	**Faire attaquer les objets.** Définir les six coups et faire agir un personnage sur un autre.
10:25–10:35	**Pause** (10 minutes).
10:35–11:10	**Protéger les PV et gérer l'esquive.** Vérifier les bornes et résoudre les probabilités de touche.
11:10–11:40	**Ajouter les XP.** Calculer le coefficient, puis attribuer l'expérience après un coup réussi.
11:40–12:10	**Spécialiser les combattants.** Héritage, `super()` et présentation polymorphe.
12:10–12:20	**Pause** (10 minutes).
12:20–13:00	**Assembler le combat.** Choix de l'attaque, tour du Roi Mort, arrêt et vainqueur dans `start()`.
13:00–13:20	**Finaliser le TP.** Affichage des états et tests des règles principales.
13:20–13:30	**Démontrer et vérifier.** Exécuter la partie complète, corriger les derniers défauts.

Rendu attendu : les six fichiers de l'arborescence, avec un programme qui démarre par `main.py`, un combat jouable jusqu'à la victoire et un fichier de tests exécutable. Les diagrammes UML sont fournis comme spécification ; il n'est pas demandé de les redessiner.

11 / VALIDATION FINALE

Avant de remettre votre jeu

Utilisez cette grille pour vérifier vous-même que votre application est complète, avant de la montrer au formateur.

- ☐ **Lancement.** J'exécute `main.py` et c'est bien `CombatManager.start()` qui lance la partie.
- ☐ **Modèle objet.** J'ai une classe mère `Personnage`, deux classes filles, et une classe `CombatManager`.
- ☐ **Attaques.** Gandalf et le Roi Mort ont chacun exactement trois coups conformes au barème.
- ☐ **Choix.** Je peux choisir 1, 2 ou 3 ; toute autre entrée est refusée sans casser le programme.
- ☐ **Esquive.** Les probabilités sont prises en compte et un coup esquivé n'a aucun effet sur les PV ou les XP.
- ☐ **Progression.** Un coup réussi donne 10 XP ; le coefficient augmente pour le coup suivant et reste au plus égal à ×2.
- ☐ **Robustesse.** Les PV ne sont jamais négatifs et un personnage vaincu ne peut ni attaquer ni subir de nouvelle attaque.
- ☐ **Victoire.** Le combat s'arrête dès qu'un personnage tombe à 0 PV ; le gagnant est annoncé.
- ☐ **Tests.** Je peux démontrer les cas de réussite, d'esquive, de bonus XP et de coup final sans riposte.
- ☐ **Lisibilité.** Chaque fichier et chaque méthode possèdent une responsabilité compréhensible.

Notions POO réinvesties

Classe et objet · Constructeur __init__ · self · Méthodes · Encapsulation · @property · Héritage · super() · Redéfinition · Polymorphisme · Collaboration entre objets

> **Critère essentiel :** le projet n'est pas seulement un jeu qui affiche un vainqueur. Vous devez pouvoir expliquer pourquoi les règles communes sont dans `Personnage`, pourquoi les attaques propres sont dans les classes filles et pourquoi les tours appartiennent à `CombatManager`.

Document étudiant · Support de réalisation accompagné · Version alignée sur le projet `CombatManager` V2.